ACML tutorial: Artificial Neural Networks

Advanced Concepts in Machine Learning — Assignment 1

In this lab, you will implement a neural network. The implementation can be made in any language of your choice: Java, MATLAB, Octave, Python, Prolog (for the extra challenge) ... your choice, but you can not (yet) rely on high-level libraries such as Keras, PyTorch, Theano, Tensorflow, Deeplearning4J, the MATLAB Neural Network Toolbox, etc. You need to implement your own backpropagation for this assignment. You should solve this assignment in pairs. (Hint: Things get easier when you vectorise.)

Task specification

Network architecture

The goal of the implementation is to produce and train a network with 3 layers: input - hidden - output. Both the input layer and the output layer will have 8 nodes, the hidden layer only 3 nodes (+biases).

Learning examples

The learning examples will each have 7 zeros and 1 one in them (so there will be only 8 different learning examples, and you will have to repeat them), and the output the network should learn is exactly the same as the input. So when the input layer is given $< 0, 0, 0, 1, 0, 0, 0, 0 >$ as input, the output to aim for is also $< 0, 0, 0, 1, 0, 0, 0, 0 >$.

Try to get your network to learn this reproducing function on the 8 different learning examples.

Then study the activations of the hidden nodes of your network and try to interpret them.

Deliverables

1. A runnable demo

A runnable (compilable and runnable) demo of your network. This means including any extra files such as learning examples if needed.

2. A short report

A short report where you

1. describe your software and how to use it (like a README file);
2. give a brief description of the learning performance of your network (how many examples does it need to converge, how long does that take, does it converge every time or do you have to be lucky with the parameters[a], etc.); and
3. give your interpretation of the learned weights.

[a] Since this was apparently not obvious in the past, this implies actually trying different parameter values.

Handing in

Deadline and submission rules

To hand in, register both students as part of an assignment pair and upload your[a] zipped code, report (in PDF format only) and **a short (2-5 minutes) video** tracing your code and report separately (so 3 different files, namely zip (with the code), PDF and video) through the student portal. Late **or partial** submissions (missing video, code, and/or report) will **not** be graded (you will not get credit for your work). One submission per pair is sufficient, though both students may submit if preferred. Each submission must include the partner's name **both in the report and** as a comment on the submission.

[a]There are probably more NN implementations available on the internet than I can count on my 10 binary fingers. You can, of course, use these for inspiration, but please do not steal an implementation and turn this in as your own. Not only is this in violation of our rules, but you will learn so much less doing this, and you (or someone else on your behalf) are paying for you not to learn as much as you possibly can.

Note: A Jupyter Notebook that combines code and report can also be submitted. However, you must submit not only the *ipynb* file but also a PDF version. Therefore, the three files to be submitted will be an *ipynb*, a PDF and the video.

Hint on the video preparation

Tip: how to spend your five minutes

The video must be short, we will stop checking it after minute 5:00. To decide what to prioritise in the video, we highly recommend that you base your explanation on the rubric provided within the assignment in Canvas. The video can be prepared by one or both students.